

SPS3 — Socket Programming: Protocols

Beginner Exam Cheat Sheet

1 · Network Protocols — The Basics

A **protocol** is a set of rules that two parties must follow to communicate. You need protocols at *every layer* of the network stack. TCP and IP are protocols. After a TCP connection is established, the two processes talk at the **application layer**.

At the application layer, parties must agree on:

- Who speaks first?
- How much data can each side send at once?
- How are messages **encoded into bytes**?

2 · Byte Streams — How Data Travels

A socket gives your application a **byte stream**. It is up to *you* to decide what those bytes mean. Both sides must agree on the same encoding scheme.

Option A — Plain Text (XML / JSON)

Data is sent as human-readable ASCII text.

- Easy to debug (readable in Wireshark)
- Needs **parsing** on the receiving end
- Each line ends with a delimiter (e.g. `\n` or `\r\n`)

Sending integer 128 as text:

```
send_str("128\n"); // chars: '1','2','8','\n'
```

Option B — Raw Binary

Data is sent as raw bytes (fast, compact).

- No parsing needed
- Must handle **endianness**
- Must handle **struct padding**

Sending integer 128 as 4 bytes:

```
// 0x00 0x00 0x00 0x80 (big-endian)
```

3 · Endianness — Byte Order

When a value is larger than 1 byte, machines differ in which byte they store first. All parties must agree, or the receiver will interpret the number incorrectly.

Term	Meaning	Example (0x80290CD9 = 2,150,173,913)
Big-Endian	Most significant byte first (network standard)	80 29 0C D9
Little-Endian	Least significant byte first (x86 default)	D9 0C 29 80
Network Byte Order	Always big-endian — used on the wire	<i>Always convert before send!</i>

4 · Byte Order Conversion Functions (MUST KNOW)

Rule: Convert to network byte order BEFORE sending. Convert to host byte order AFTER receiving.

Function	Bits	Direction	Use When...
----------	------	-----------	-------------

htonl()	32-bit	Host → Network	Sending an int
ntohl()	32-bit	Network → Host	Receiving an int
htons()	16-bit	Host → Network	Sending a short
ntohs()	16-bit	Network → Host	Receiving a short

Header to include:

```
#include <arpa/inet.h>;
```

Example usage:

```
int val = 128;

// Before sending:

uint32_t net_val = htonl(val);

send(fd, &net_val, sizeof(net_val), 0);

// After receiving:

uint32_t received;

recv(fd, &received, sizeof(received), 0);

int host_val = ntohl(received);
```

5 · Strings & Character Encoding

In this course you can assume **ASCII** strings (each character = 1 byte, no endianness issue). Real-world apps use **Unicode / UTF-8**.

Python 3 string encoding example:

```
msg = "hello"

byte_data = msg.encode("utf-8") # str → bytes

back = byte_data.decode("utf-8") # bytes → str
```

6 · Struct Padding & Alignment

The compiler may insert **invisible padding bytes** inside structs to align fields. Two machines (even both Intel) can produce different layouts if one is 32-bit and the other 64-bit.

■ Dangerous — implicit padding

```
struct RecTag {

    int a; // 4 bytes

    short b; // 2 bytes

    // ← 2 padding bytes inserted!

    int c; // 4 bytes

};
```

■ Safe — explicit padding

```
struct RecTag {

    int a; // 4 bytes

    short b; // 2 bytes

    char unused[2]; // explicit pad

    int c; // 4 bytes

};

// Also convert a, b, c with htonl/htons!
```

Fix strategies: (1) Add explicit padding fields, (2) Reorder fields from largest to smallest, (3) Use a serialisation library (e.g. JSON) to avoid raw structs entirely

7 · Command Encoding — 3 Options (TFTP-style)

When designing a simple file-transfer protocol (like TFTP), you must choose how to encode commands. The three main options are **text**, **fixed-size binary**, and **dynamic binary**.

Option A · Text Commands (delimiter-based)

```
// Client sends: "get hello.txt\n"
send_str(sid, "get hello.txt\n");

// Server reads until '\n'
recv_line(sid, buf, BUF_SIZE);
```

PRO	CON
✓ Human-readable / easy to debug	✗ Requires parsing
✓ Flexible, compatible	✗ Variable length — must check for delimiter

Option B · Fixed-Size Binary Commands

```
#define LS 0
#define GET 1
#define EXIT 3

struct Command {
    int code; // 4 bytes
    char fileName[252]; // fixed buffer
} cmd;

cmd.code = htonl(GET); // convert byte order!
strncpy(cmd.fileName, "hello.txt", 252);
send_all(fd, &cmd, sizeof(cmd));
```

PRO	CON
✓ No parsing needed	✗ Hard to read & maintain
✓ Fixed / predictable size	✗ Wastes bytes (all cmds same size)

Option C · Dynamic-Size Binary Commands

```
// Send: [4-byte code][4-byte name_length][name bytes]

int code = htonl(GET);
int fnlen = strlen(fname);
int nlen = htonl(fnlen);
```

```

send_all(fd, &code, sizeof(int));

send_all(fd, &nlen, sizeof(int));

send_all(fd, fname, fnlen); // no NUL needed

```

PRO	CON
✓ No parsing	✗ Harder to implement
✓ Sends only necessary bytes	✗ Endianness on every field

8 · Response Header Pattern

A common design sends a fixed-size **header** first (describing the payload), then the variable-length **payload**.

```

typedef struct {
    int code; // e.g. 0=text, 1=file
    int length; // number of bytes that follow
} ResponseHeader;

// SEND SIDE

ResponseHeader hdr;
hdr.code = htonl(Resp_FILE);
hdr.length = htonl(file_size);
send_all(fd, &hdr, sizeof(hdr));
send_all(fd, file_buf, file_size);

// RECEIVE SIDE

ResponseHeader hdr;
recv_all(fd, &hdr, sizeof(hdr));
int code = ntohl(hdr.code);
int len = ntohl(hdr.length);
recv_all(fd, payload_buf, len);

```

9 · Protocol Walkthrough — SMTP

SMTP (Simple Mail Transfer Protocol) is a *text-based* protocol. Every line ends with **\r\n** (CR+LF). Server responses start with a 3-digit code.

Server	220 smtp.example.com ESMTP Postfix
Client	HELO relay.example.com
Server	250 smtp.example.com, I am glad to meet you
Client	MAIL FROM:<bob@example.com>
Server	250 Ok
Client	RCPT TO:<alice@example.com>

Server	250 Ok
Client	DATA
Server	354 End data with <CR><LF>.<CR><LF>
Client	From: Bob To: Alice Subject: Test
Client	Hello Alice!
Client	. ← single dot on its own line = end of DATA
Server	250 Ok: queued as 12345
Client	QUIT
Server	221 Bye {server closes connection}

10 · Safe String Handling in C (EXAM TRAP!)

Unsafe functions (gets, strcpy, scanf("%s" ...)) do NOT check buffer size → buffer overflow! Always use the safe alternatives below.

■ UNSAFE — Never Use	■ SAFE — Always Use	Notes
gets(buf)	fgets(buf, sizeof(buf), stdin)	fgets includes the \n
strcpy(dst, src)	strncpy(dst, src, n)	strncpy may miss NUL — add it manually
strcat(dst, src)	strncat(dst, src, n)	n = remaining space, not total
scanf("%s", buf)	scanf("%10s", buf)	Number limits input width
sprintf(buf, "%s", s)	snprintf(buf, sizeof(buf), "%s", s)	snprintf never overflows
strlen(s)	strnlen(s, n)	Safe if s is not NUL-terminated

11 · Quick Reference & Exam Tips

Always convert integers	Use htonl/ntohl (32-bit) or htons/ntohs (16-bit) before every send/receive of multi-byte integers.
Check every return value	send(), recv(), read(), write() can return less than you asked for. Loop until all bytes sent/received.
Delimiters in text protocol	SMTP uses \r\n. Simple protocols often use \n. You must scan the stream for the delimiter.
Fixed struct binary	All commands same size — easy receiver, but wastes bandwidth on short commands.
Dynamic binary	Send length first, then data. Receiver reads length, then reads exactly that many bytes.
No implicit NUL in network strings	When you send a string as raw bytes you do NOT need to send the NUL terminator (include only the real chars).
JSON/XML = text option	Human-readable, language-agnostic, self-describing — but needs a parser and is bulkier.
Struct padding = silent bug	Add explicit padding or use JSON to avoid architecture-specific layout differences.

12 · TFTP-Style Protocol — Command Summary

Three commands supported in the example TFTP-style protocol:

Command	Code	Client Sends	Server Responds With
---------	------	--------------	----------------------

LS	0	Just the command code	List of filenames
GET fn	1	Command code + filename	File contents (bytes)
EXIT	3	Just the command code	Nothing — connection closes